

The Scenario

Your task is to build a simple **uptime monitor**: a lightweight, full-stack application that periodically pings a list of URLs and displays whether each one is active, along with its response time.

Because your background leans heavily toward core infrastructure and backend systems, components like building a frontend UI may not be part of your day-to-day workflow. That is the core point of this exercise. We want to see you stretch across the stack and lean entirely on your choice of AI assistants (e.g., Cursor, Claude, Copilot) to rapidly generate, debug, and connect the layers you do not write on a daily basis.

Context & Scale

- **Scale:** Strict MVP. A few dozen monitored URLs, checked every minute or so. No massive scale overhead is required here.
- **Pragmatism:** Keep it beautifully simple. As an early-stage startup, we highly value execution velocity and clean, working, end-to-end applications over complex, over-engineered architectures.

Core Components

1. Backend API

An API (using any language or framework of your choice) that allows a user to register a URL, periodically pings each registered URL, and stores the HTTP status code, response time, and timestamp of each individual health check.

2. Frontend UI

A simple, functional interface that lists the monitored URLs with their current operational status (up/down) and latest response times. It does not need to be visually flawless—it simply needs to work and reflect real-time data dynamically.

3. Containerization

The entire environment must be orchestrated to spin up locally out-of-the-box with a single `docker compose up` command.

4. Deployment Sketch (Light)

Include a short note in your README—or a brief, hypothetical Terraform or Pulumi snippet—describing how you would deploy this MVP application to a cloud provider. Keep this brief; we are not grading production hardening or security policies here, we just want to observe how you approach cloud topologies.

 **Verifying Your Code:** Point your monitor at a healthy URL (e.g., `[https://example.com](https://example.com)`) and an unreachable or invalid one to

demonstrate that your logic correctly detects and renders both "up" and "down" states. Document the exact steps to reproduce this test in your README so our team can instantly verify it locally.

Submission Format

Please provide a link to a **GitHub repository** containing your solution. Your repository must be structured around the following three core deliverables:

1. Repository Structure

The repository must contain the fully working, end-to-end codebase of your MVP, organized cleanly so it can be spun up via a single command line:

- `/backend` – The metric logging and pinging API.
- `/frontend` – The status and monitoring dashboard.
- `docker-compose.yml` – Orchestrating your local multi-container network.

2. Dedicated AI Collaboration Log (**AI_LOG.md** or a section in the README)

This is the most critical component of the assignment for Sumit and our technical team. Please provide a brief "peek behind the curtain" of how you collaborated with AI to build this application, including:

- **The AI Tech Stack:** Specify which AI tools and underlying LLMs you leveraged (e.g., Cursor, Claude 3.5 Sonnet, GitHub Copilot).
- **The Prompts that Shipped It:** Please share the raw interactions or specific text prompts you fed to your AI assistant to generate the core backend framework and frontend UI layers.
- **The Course Corrections:** A short note detailing an instance where the AI generated bad code, hallucinated a library, or proposed a broken architecture, and how you explicitly prompted it or refactored the code to resolve the block. - you can use your raw interaction as the base and point this out.

3. System Verification (**README.md**)

Your project README must clearly highlight:

- **A 1-Line Setup:** The exact `docker compose up` command to launch the entire ecosystem locally.
- **Testing Steps:** Transparent instructions showing our engineering team how to add a working URL and an intentionally broken URL to verify that the up/down state tracking behaves correctly under evaluation.
- **The Deployment Sketch:** Your brief architectural note or small, hypothetical Infrastructure-as-Code (IaC) block mapping out how you would host this system on a cloud provider.

